Лабораторная работа №1. Radix Tree

Описание Radix Tree

Radix Tree (базисное дерево) — это структура данных, позволяющая эффективно хранить ассоциативный массив, ключами которого являются строки.

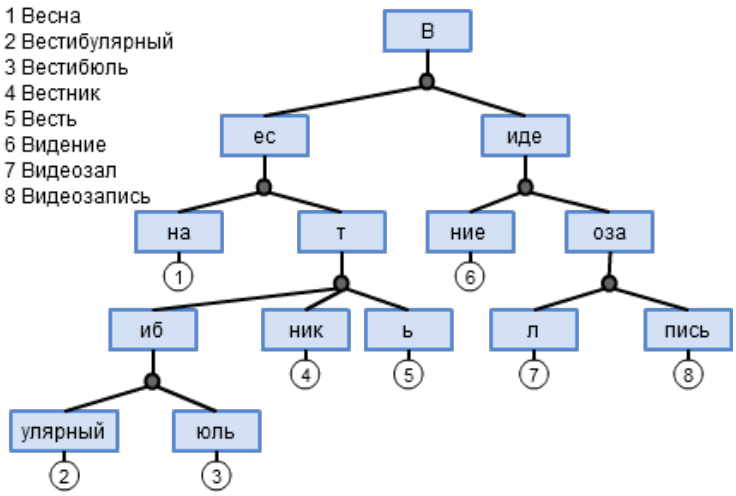


Рис. 1. Пример базисного дерева

Каждый узел дерева ассоциируется с подстрокой — его меткой. Будем полагать, что метка кореня дерева — пустая строка, чтобы было возможно хранить строки, начинающиеся на разные символы.

Рассмотрим основные операции над этой структурой данных.

Проверка наличия строки

Допустим, необходимо найти строку s. Обход начинается с корня. Для текущего узла v ищется дочерний узел v_c, соответствующий следующей подстроке строки s. Если такого узла нет, то строка s в дереве отсутствует. Если такой узел найден, то необходимо сравнить размер его метки с размером ненайденной части строки s. Возможны следующие ситуации:

1. Если метка длиннее, то строка s в дереве отсутствует.
2. Если размеры совпадают и узел v_c — лист или он указывает на метку как на возможный конец (например, с помощью пустого дочернего узла), то строка s в дереве присутствует (как если бы в дереве на картинке было слово «видео» и требовалось его найти).
3. Если ненайденная часть строки s длиннее и у v_c есть дочерние узлы, то в качестве текущего узла выбираем его ($v \leftarrow v_c$) и повторяем процесс. Иначе строка s в дереве отсутствует.

Здесь можно найти более детальное описание этого алгоритма.

Добавление строки

Допустим, необходимо вставить строку s. Для этого производится её поиск в дереве. Если метка узла, на котором поиск остановился, имеет общее начало со остатком строки s, она разбивается на две части и добавляются узлы с отличающимися частями метки и остатка s.

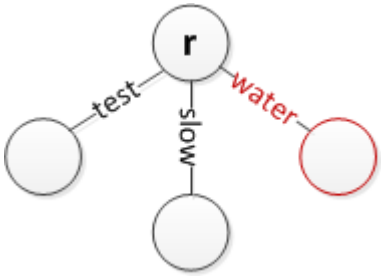


Рис. 2. Вставка "Water" в корень

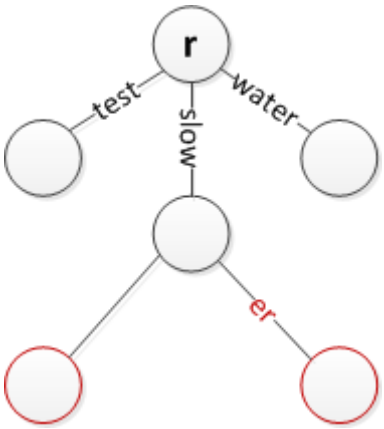


Рис. 3. Вставка "slower" в корень

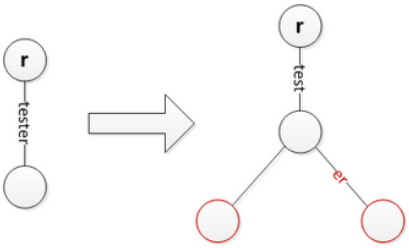


Рис. 4. Вставка "test", который имеет общее начало с "tester"

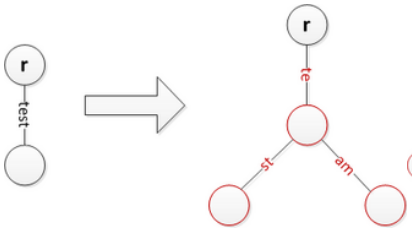


Рис. 5. Вставка "team" с разбиением "test" на "te" и "st"

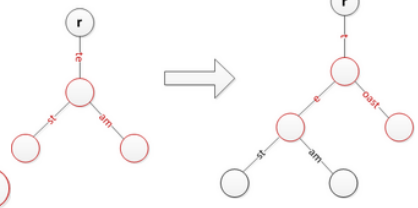


Рис. 6. Вставка "toast" с разбиением "te" и сдвигом узлов на уровень ниже

Картинки позаимствованы из Википедии.

Удаление строки

Допустим, необходимо удалить строку *s*. Для этого находится соответствующий лист, а затем удаляется. Если после этого у родительского элемента остаётся только один дочерний элемент, то дочерняя метка добавляется к родительской метке, а дочерний узел тоже удаляется.

Поиск всех строк с общим началом

На вход подаётся префикс *s*. Производится поиск данной строки в дереве. Если для префикса подходящий узел существует, то с помощью обхода поддерева, корнем которого он является, осуществляется конкатенация меток в отдельные строки.

Задача

Реализуйте класс `RadixTree` со следующими публичными методами:

1. `contains(...)` — метод, проверяющий наличие строки в дереве;
2. `insert(...)` — метод, вставляющий строку в дерево;
3. `delete(...)` — метод, удаляющий строки из дерева;
4. `get_by_prefix(...)` — метод, который возвращает набор строк, начинающихся на заданный префикс;
5. `size()` — метод, возвращающий количество строк в дереве;
6. `empty()` — метод, проверящий дерево на пустоту;
7. конструктор по умолчанию;
8. конструктор с `std::initializer_list<std::string>`, который инициализирует дерево набором строк;
9. конструктор копирования;
10. деструктор;

11. перегрузка оператора << для вывода содержимого дерева (формат не важен);
12. оператор присваивания копированием;
13. оператор ==, проверяющий деревья на заполнение одинаковыми элементами.

Всего за работу можно получить 13 баллов. Пункты 1–3 оцениваются в 3 балла. Пункт 4 — в 2 балла.

Используйте класс `std::string` из стандартной библиотеки в качестве строк. Может оказаться полезным и его невладеющий аналог — `std::string_view`.

Каждый публичный метод должен быть покрыт тестами.

Инструменты

Windows

Самый простой способ получить компилятор C++ — это установка Visual Studio с рабочей нагрузкой «Разработка классических приложений на C++». Для совместимости с шаблоном проекта также необходимо подключить нагрузку «Linux, Mac и встроенная разработка на C++». Она включает в себя систему сборки CMake, которая будет доступна только из Visual Studio.

Другой вариант — установка mingw-w64 с CMake и Ninja. В качестве редактора кода подойдёт Visual Studio Code. Для статического анализа и автодополнения лучше всего использовать расширение clangd (при установке нужно согласиться на докачивание пакетов), т.к. расширение «C/C++» от Microsoft с IntelliSense работает медленно и автоматически не подхватывает зависимости из проекта. Расширение CMake Tools добавляет кнопки на нижней панели для быстрой сборки проекта и запуска целей, а также позволяет выбрать компилятор.

Третий вариант — подключение WSL, подсистемы Windows для Linux. Для неё инструменты аналогичны инструментам под Linux.

Linux/MacOS

Связка gcc + CMake + Visual Studio Code + расширения clangd и CMake Tools должна работать без проблем.

Шаблон проекта

Для удобства был создан шаблон проекта, использующий CMake. Он содержит директории:

- `include/` — содержит заголовочные файлы. Именно в них вам нужно прописать классы и объявления их методов.
- `src/` — содержит файлы исходного кода. В них стоит помещать определения больших функций и методов.
- `tests/` — содержит тесты. В проект подключена библиотека catch2, предназначенная для написания тестов.